T5 - Pandas and NumPy: load, clean, summarise, write

Covers NumPy arrays, pandas CSV loading, missing values, dates, groupby and writing results. Earns the marks of 2024 Task 1a (9 of 13) and the pandas/NumPy half of "Practice Exam Task 3" (statistics + summary file); it also feeds every chart in T6.

1. What the exam asks

2024 final lab exam, Python Task 1 [13 marks] - file `sales_data.csv` (columns `Date, Product, Quantity, Price, Total`):

> **Task 1a: Data Loading and Preprocessing** - Load the CSV file into a Pandas DataFrame. [2 mark] - Ensure there are no missing values. If any are found, fill them with appropriate values. [3 mark] - Convert the Date column to a datetime object. [2 mark] - Ensure the Total column correctly represents the product of Quantity and Price. If any discrepancies are found, correct them. [2 mark]

Practice Exam Task 3 (replaces PyTorch) - Task 3 "Summary Statistics":

> After saving the data, use pandas and Numpy to: read the CSV file into a DataFrame; print the total number of records; print the average, minimum, and maximum heights; save summary results to a text file called `height_summary.txt`.

(The heights *logger* - appending rows, `q` to quit - is in the T3 document.)

Where the course taught this: `WEEK3/Class4/Data_Manipulation.ipynb` (the shopping.csv walk-through), the slides "More Data Manipulation - date format and missing data" (slides 3-9: `parse_dates`, `dayfirst`, `isnull().sum()`, `fillna({...})`), "Pandas-reading and writing", the NumPy masking slides, and `WEEK3/Lab04` (`Lab4_solutions.pdf` Tasks 1-13, `Lab 4_2` indexing exercises). The 2022 and 2023 papers had no pandas; 2024 is the pattern to expect.

What the 2024 data really contains (checked on the real file): 151 rows; dates are text like `01/01/2023` (dd/mm/yyyy); years 2022 (29 rows), 2023 (100), 2024 (22); 6 blank `Product`, 1 blank `Price`, 3 blank `Total`; no blank `Date` or `Quantity`; every non-blank Total already equals Quantity x Price.

2. Mental model (analogy)

- **DataFrame = a spreadsheet** held in memory. **Series = one column** of it. `df['Price']` hands you that column.
- **NaN = an empty cell.** `isna()` ticks the empty cells; `fillna` types a value into them; `dropna` throws away the rows that contain one.
- **NumPy array = a row of numbered lockers**, all holding the same type of thing. `a[2]` opens locker 2; `a[1:4]` opens lockers 1,2,3.
- **Boolean mask = a stencil.** `a > 25` makes a True/False sheet the same size as `a`; `a[mask]` lays the stencil over and keeps only the True holes.
- **groupby = sorting receipts into piles** (one pile per product), then adding up (`sum`) or averaging (`mean`) each pile.
- `.dt.to_period('M')` **= tearing the date off and keeping only the calendar page** (2023-01 instead of 2023-01-17) so all January receipts share a label.
- `axis`: `axis=0` = squash *down the rows* (one answer per column); `axis=1` = squash *across the columns* (one answer per row).

3. Copy-paste templates

All templates assume `import numpy as np` and `import pandas as pd`. The CSV must sit in the same folder as your `.py` file (or give a full path).

A. NumPy: create, shape, index, slice, mask, fancy, aggregate (use for Lab4-style questions; sources `Lab 4_2 ...docx`, `Lab4_solutions.pdf`)

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])            # 1-D from a list
m = np.arange(1, 10).reshape(3, 3)            # 1..9 as a 3x3 grid
print(a.shape, m.shape, m.ndim, a.dtype)      # (5,) (3, 3) 2 int64
z = np.zeros(3); o = np.ones((2, 2)); f = np.full(3, 5)   # starter arrays
r = np.arange(0, 10, 2)                       # [0 2 4 6 8]  stop excluded

print(a[0], a[-1], a[1:4], a[::2])            # 10 50 [20 30 40] [10 30 50]
print(m[1, 2], m[0], m[:, 0])                 # 6  [1 2 3]  [1 4 7]
print(m[0:2, 1:3])                            # rows 0-1, cols 1-2
print(m[-2:], m[:, :2])                       # last two rows / first 2 cols

mask = a > 25                                 # [F F T T T]
print(a[mask], a[(a > 15) & (a < 45)])        # [30 40 50] [20 30 40]
b = a.copy(); b[b <= 20] = 0                  # change values using a mask
print(a[[0, 2, 4]])                           # fancy index: [10 30 50]
g = np.arange(1, 17).reshape(4, 4)
print(g[[0, 1, 2, 3], [0, 2, 3, 1]])         # pairs (0,0)(1,2)(2,3)(3,1)

print(m.sum(), m.mean(), m.min(), m.max())    # 45 5.0 1 9
print(m.sum(axis=0), m.sum(axis=1))           # [12 15 18]  [ 6 15 24]
print(np.where(a > 25, 'big', 'small'))       # if/else for every element
```

B. Load a CSV and look at it first (always the first lines)

```
df = pd.read_csv('sales_data.csv')   # CSV -> DataFrame
print(df.head())                     # first 5 rows (head(10) for 10)
print(df.shape)                      # (151, 5) = (rows, columns)
print(list(df.columns))              # ['Date', 'Product', ...]
df.info()                            # dtypes + non-null counts (no print!)
print(df.describe())                 # count/mean/std/min/max of numbers
print(df.dtypes)                     # type of each column
print(df['Product'].unique())        # distinct values (NaN shows up)
```

C. Select columns, rows and filter (boolean filter = the mask idea again)

```
prices = df['Price']                          # one column (a Series)
two = df[['Product', 'Total']]                # several columns: DOUBLE []
print(df.loc[0, 'Product'])                   # by label: row 0, column name
print(df.loc[0:2, ['Product', 'Total']])      # loc slice INCLUDES row 2
print(df.iloc[0, 1], df.iloc[:2, :2])         # by position; end EXCLUDED
big = df[df['Quantity'] > 8]                  # rows where condition True
both = df[(df['Quantity'] > 5) & (df['Price'] > 10)]   # & | with brackets
some = df[df['Product'].isin(['Water', 'Sweets'])]     # any of a list
blank = df[df['Product'].isna()]              # rows with a blank Product
print(len(big), big['Total'].sum())           # how many / their total
```

D. Missing values: find, fill, drop, de-duplicate, change type

```
print(df.isna().sum())               # blanks per column (isnull() = same)
print(df.isna().sum().sum())        # total blanks in the whole table

d = df.copy()                        # experiment on a copy
d['Price'] = d['Price'].fillna(d['Price'].mean())        # fill with mean
d['Total'] = d['Total'].fillna(d['Total'].median())      # ... or median
d['Product'] = d['Product'].fillna(d['Product'].mode()[0])   # most common
d = df.fillna({'Product': 'Unknown', 'Quantity': 0,
               'Price': 0.0, 'Total': 0.0})   # dict form: one value/column
print(d.isna().sum().sum())          # 0
```

```
clean = df.dropna()                          # drop rows with ANY blank
clean = df.dropna(subset=['Product'])        # ... only if Product is blank
nodup = df.drop_duplicates()                 # remove exact duplicate rows
d['Quantity'] = d['Quantity'].astype(float)  # change a column's type
d['Quantity'] = d['Quantity'].astype(int)    # float -> int (no NaN allowed)
```

E. Dates: text -> datetime -> year / month / period

```
dd = df.copy()
dd['Date'] = pd.to_datetime(dd['Date'], dayfirst=True)       # dd/mm/yyyy
dd['Date'] = pd.to_datetime(df['Date'], format='%d/%m/%Y')   # strict form
dd['Date'] = pd.to_datetime(df['Date'], dayfirst=True,
                            errors='coerce')   # bad text -> NaT, not a crash
dd['Year'] = dd['Date'].dt.year              # 2023
dd['Month'] = dd['Date'].dt.month            # 1..12
dd['Period'] = dd['Date'].dt.to_period('M')  # 2023-01 (year+month)
dd['Weekday'] = dd['Date'].dt.day_name()     # 'Sunday'
y23 = dd[dd['Date'].dt.year == 2023]         # keep only 2023 rows
print(dd.dtypes['Date'], len(y23))           # datetime64[..] 100
```

F. New columns, groupby, sort, count, apply (the "analysis" toolbox)

```
dd['Total'] = dd['Quantity'] * dd['Price']          # (re)compute a column
dd['Big'] = dd['Quantity'].apply(lambda q: 'big' if q >= 8 else 'small')
dd['Band'] = np.where(dd['Total'] > 100, 'High', 'Low')   # vectorised if/else

per_prod = dd.groupby('Product')['Quantity'].sum()  # Series: product -> total
avg_tot = dd.groupby('Product')['Total'].mean()     # mean per product
n_rows = dd.groupby('Product').size()               # rows per product
table = dd.groupby('Product')['Total'].agg(['sum', 'mean', 'count'])
per_year = dd.groupby(['Year', 'Product'])['Quantity'].sum()   # two keys

print(per_prod.sort_values(ascending=False))        # biggest first
print(dd.sort_values('Total', ascending=False).head(3))   # top 3 rows
print(dd['Product'].value_counts())                 # how many of each
print(per_prod.idxmax(), per_prod.max())            # best product + its value
print(dd['Total'].sum(), round(dd['Total'].mean(), 2))
```

G. EXAM Task 1a - paste-ready (tested on the real `sales_data.csv`)

```
import pandas as pd

# ---------- Task 1a: load, clean, dates, Total ----------
df = pd.read_csv('sales_data.csv')             # 1. load the file  [2]

# 2. no missing values  [3]
# Price blank but Total known -> recover Price = Total / Quantity
rec = df['Price'].isna() & df['Total'].notna() & (df['Quantity'] > 0)
df.loc[rec, 'Price'] = df['Total'] / df['Quantity']
# whatever is still blank gets a sensible default (one value per column)
df = df.fillna({'Product': 'Unknown', 'Quantity': 0,
                'Price': 0.0, 'Total': 0.0})

# 3. Date text (dd/mm/yyyy) -> datetime  [2]
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)

# 4. Total must be Quantity x Price  [2]
df['Total'] = df['Quantity'] * df['Price']     # recompute whole column

print(df.isna().sum().sum(), 'blanks left')    # expect 0
print(df.dtypes)
print(df.head())
```

Why a *fill strategy per column* (examiners ask "appropriate values"):

Column	Blank means	Fill with	Reason
Product	a text label is unknown	`'Unknown'`	keeps the sale; never use 0 in a text column

Column	Blank means	Fill with	Reason
Price	price not recorded	`Total / Quantity` first, else `0.0`	the data lets us recover it exactly (row 70: 50/5 = 10)
Total	total not recorded	`Quantity * Price`	it is a derived column - step 4 does this for everyone
Quantity	(none blank here)	`0`	defensive default in case the exam file differs
Date	(none blank here)	do NOT invent; `errors='coerce'` then `dropna(subset=['Date'])`	a made-up date corrupts the monthly charts

TIP: the course solution (`ProgSD-Python-Solution.py`) fills Price with `0.0` and then recomputes Total. That is accepted, but it turns the row with the blank Price (Chocolate, 12/03/2023, Total 50) into Total = 0 and under-reports March 2023 sales by 50. Recovering the Price first is strictly better and costs three lines.

H. Writing results to files (Practice Exam Task 3, step 3)

```
heights = pd.DataFrame({
    'Name': ['Amy', 'Ben', 'Ali', 'Yin', 'Mohammed', 'Gus'],
    'Height': [165, 172, 166, 180, 170, 182]})
heights.to_csv('heights.csv', index=False)     # index=False: no 0,1,2 column

h = pd.read_csv('heights.csv')                 # read it back
count = len(h)                                 # number of records (6)
avg = h['Height'].mean()                       # or np.mean(h['Height'])
low, high = h['Height'].min(), h['Height'].max()
print('Records:', count)
print(f'Average: {avg:.1f}  Min: {low}  Max: {high}')   # 172.5 165 182

with open('height_summary.txt', 'w') as f:     # text report, closed for us
    f.write(f'Total records: {count}\n')
    f.write(f'Average height: {avg:.2f} cm\n')
    f.write(f'Minimum height: {low} cm\n')
    f.write(f'Maximum height: {high} cm\n')

df.to_csv('cleaned_sales.csv', index=False)    # save the cleaned sales too
df['Date'].dt.strftime('%Y-%m-%d')             # datetime -> 'YYYY-MM-DD' text
                                               # (what SQLite wants, see T4)
```

4. Line-by-line walkthrough (Task 1a)

```
import pandas as pd                       # load the spreadsheet library
df = pd.read_csv('sales_data.csv')        # open file; row 1 = column names
# df is now the "spreadsheet". Dates are still just TEXT here.

# a True where Price is empty AND Total is known AND we can divide by Qty
rec = (df['Price'].isna()                 # empty price cells ...
       & df['Total'].notna()              # ... that have a Total ...
       & (df['Quantity'] > 0))            # ... and a safe divisor
# .loc[rows, column] = "write into THESE rows of THIS column"
df.loc[rec, 'Price'] = df['Total'] / df['Quantity']   # Price = Total / Qty

# dict form: column -> value to type into its empty cells
df = df.fillna({'Product': 'Unknown',     # text column gets text
                'Quantity': 0,            # counts: 0
                'Price': 0.0,             # leftover blank prices
                'Total': 0.0})            # (step 4 overwrites these)

# text '13/01/2023' -> real date; dayfirst=True says 13 is the DAY
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)

# whole-column arithmetic: row-by-row, no loop needed
df['Total'] = df['Quantity'] * df['Price']
```

Same idea in the course's own idiom (`Data_Manipulation.ipynb`: parse dates while loading). Both work; pick ONE:

```
df2 = pd.read_csv('sales_data.csv', parse_dates=['Date'], dayfirst=True)
print(df2['Date'].dtype)                  # datetime64[us] = it worked
```

Which date form? `pd.to_datetime(df['Date'], dayfirst=True)` is the safer habit: it raises an error if parsing fails, so you notice. With `parse_dates=['Date']` and *no* `dayfirst`, pandas 3 does not complain - it silently leaves the column as text (dtype `str`), and the next `.dt` line crashes. Always pair `parse_dates` with `dayfirst=True` for dd/mm/yyyy.

Month column for charts (the course's idiom, slide 6 of the dates deck):

```
df['Month'] = df['Date'].dt.to_period('M')        # 2023-01, 2023-02 ...
monthly = df.groupby('Month')['Total'].sum()      # Series indexed by month
print(monthly.head(3))                            # 2022-03 ... 2022-04 ...
```

NOTE: this groups ALL years together (2022-03 ... 2024-04). If the question says "months of 2023" filter the year FIRST (see T6).

5. Traps & debugging

Tested in this environment (Python 3.14, **pandas 3.1**, NumPy 2.5). The exam PC may have an older pandas; the code in this document avoids everything that differs, but the *messages* below can look different.

Missing-value traps

What you wrote	What happens	Fix
`df['Product'].fillna('Unknown', inplace=True)`	pandas 3: `ChainedAssignmentError` warning and **nothing changes** (6 blanks remain). Older pandas: worked, with a FutureWarning	`df['Product'] = df['Product'].fillna('Unknown')` or `df.fillna({'Product': 'Unknown'}, inplace=True)`
`df['Product'].fillna(0)`	warning "'int' is not supported as a fill value for str dtype"; column becomes a mix of text and 0	text columns get text: `'Unknown'`
`df['Price'].fillna('Unknown')`	warning "'str' is not supported as a fill value for float64 dtype"; Price turns into text and arithmetic breaks	numbers get numbers
`df.fillna('Unknown')` on the whole table	same warning; Price and Total become text columns	use the dict form
`df.fillna(method='ffill')`	`TypeError: ... unexpected keyword argument 'method'` (removed)	`df['Product'].ffill()`
`df['Price'].astype(int)` while blanks exist	`IntCastingNaNError: Cannot convert non-finite values`	fill first, then `astype`
`df['Product'] == ''` to find blanks	counts 0: blanks are read as NaN, not empty text	`df['Product'].isna()`

Date traps (the big one - the file is dd/mm/yyyy)

```
pd.to_datetime(df['Date'])        # pandas 3 -> ValueError:
  time data "13/01/2023" doesn't match format "%m/%d/%Y"
pd.to_datetime(df['Date'], errors='coerce')    # NO error, but 78 of the
  151 dates silently become NaT (every day > 12): the worst failure
df['Date'].dt.year     # AttributeError: Can only use .dt accessor with
  datetimelike values   -> the column is still text; convert it first
```

- Fix for all three: `pd.to_datetime(df['Date'], dayfirst=True)` (or `format='%d/%m/%Y'`, which never guesses).
- `errors='coerce'` is for *genuinely dirty* text only; check `df['Date'].isna().sum()` afterwards. Never add it just to hide an error.
- Older pandas may print a harmless `UserWarning` about date format inference; the result is still right.

Filtering / grouping traps

Mistake	Error you see	Fix
`df[(a > 5) and (b > 5)]`	`ValueError: The truth value of a Series is ambiguous`	use `&`, `|`, `~`, each test in brackets
`df[df['Q'] > 5 & df['P'] > 5]` (no brackets)	`TypeError: Cannot perform 'rand_'...`	`(df['Q'] > 5) & (df['P'] > 5)`
`df.groupby('Product').mean()` with text columns	`TypeError: dtype 'str' does not support operation 'mean'`	select the column: `df.groupby('Product')['Total'].mean()`
`df['product']`	`KeyError: 'product'` (case matters)	check `list(df.columns)`

Mistake	Error you see	Fix
`ydf = df[df['Quantity'] > 8]; ydf['X'] = 1`	no warning in pandas 3, and `df` is unchanged (a filter makes a copy)	assign on `df` itself, or `.copy()` deliberately
`df.loc[0:2]` vs `df.iloc[0:2]`	loc gives 3 rows (end included), iloc gives 2	remember: loc = labels, inclusive
`value_counts()` column names	pandas 3 names the counts `count`; older versions name them differently	index by position, not by name

NumPy traps

- `a[(a > 1) and (a < 3)]` -> `ValueError: truth value of an array ...` - use `&` with brackets.
- `[1, 2, 3] > 2` -> `TypeError` (a *list* is not an array; wrap in `np.array`).
- `b = a[:2]; b[0] = 99` **also changes** `a` (a slice is a view). Use `a[:2].copy()` when you need independence.
- `a + np.array([1, 2])` with different lengths -> `ValueError: operands could not be broadcast`. Shapes must match (or one must be a scalar/row).
- `a.mean()` returns `np.float64`; `print(a.mean())` is fine but a *list* of them shows `np.float64(172.5)`. Use an f-string `f'{x:.1f}'` or `float(x)`.
- `np.arange(1, 10)` stops BEFORE 10; `np.linspace(0, 1, 5)` includes both ends.

Other

- `FileNotFoundError` on `read_csv`: the CSV is not next to the `.py` file (check the working directory) or the name differs (`sales_data.csv`).
- `df.info()` already prints; writing `print(df.info())` adds a stray `None`.
- Pandas 3 shows text columns as `str`; older versions say `object`. Both mean "text".
- Never type `df.isna.sum()` - `isna` needs brackets: `isna().sum()`.

6. Anticipated exam questions

Q1 (2) Load `sales_data.csv` and show the first rows and the shape.

```
df = pd.read_csv('sales_data.csv')
print(df.head())
print(df.shape)                        # (151, 5)
```

Q2 (3) Ensure there are no missing values; fill with appropriate values and explain your choices. Use template G. Say: text -> `'Unknown'`; Price -> `Total / Quantity` where possible, else 0; Total is recomputed from Quantity x Price; do not invent dates. Prove it with `df.isna().sum()`.

Q3 (2) Convert Date to datetime and add Year and Month columns.

```
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
```

Q4 (2) Make Total equal Quantity x Price; report how many rows were wrong.

```
expected = df['Quantity'] * df['Price']            # what Total should be
wrong = ~np.isclose(df['Total'], expected)         # True = mismatch or blank
print(wrong.sum(), 'rows had a wrong Total')       # raw file: 4
df['Total'] = expected
```

Q5 (2) Which product sold the most units, and how many?

```
per_prod = df.groupby('Product')['Quantity'].sum()
print(per_prod.idxmax(), per_prod.max())       # Sweets 261
```

Q6 (3) Print the total sales for 2023 and the average sale per product for 2023.

```
y23 = df[df['Date'].dt.year == 2023]
print(y23['Total'].sum())
print(y23.groupby('Product')['Total'].mean().round(2))
```

Q7 (2) Remove duplicate rows and rows with no product, then save the result to `cleaned_sales.csv`.

```
out = df.drop_duplicates().dropna(subset=['Product'])
out.to_csv('cleaned_sales.csv', index=False)
```

Q8 (3) (Practice Task 3) Read `heights.csv`; print the number of records and the average, minimum and maximum height; save them to `height_summary.txt`. Template H.

Q9 (3) (NumPy) Given `a = np.array([12, 7, 30, 18, 25])`, print the values above 15, replace values below 10 with 0, and print the mean.

```
a = np.array([12, 7, 30, 18, 25])
print(a[a > 15])                       # [30 18 25]
a[a < 10] = 0
print(a, a.mean())                     # [12  0 30 18 25] 17.0
```

7. Quick-reference box (hand-copy this)

```
LOAD     df = pd.read_csv('f.csv')   df.head() .shape .info() .describe()
COLUMNS  df['A']   df[['A','B']]   df.loc[r,'A']   df.iloc[r,c]
FILTER   df[(df['A']>5) & (df['B']=='x')]     (& | ~ , brackets!)
BLANKS   df.isna().sum()
FILL     df = df.fillna({'Prod':'Unknown','Price':0.0})   [text->text]
         df['P'] = df['P'].fillna(df['P'].mean())
         NOT  df['P'].fillna(..., inplace=True)   (does nothing in pandas 3)
DROP     df.dropna(subset=['A'])   df.drop_duplicates()   .astype(int)
DATES    df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)  # dd/mm/yyyy
         .dt.year  .dt.month  .dt.to_period('M')
         df[df['Date'].dt.year == 2023]
NEW COL  df['Total'] = df['Quantity'] * df['Price']
GROUP    df.groupby('Prod')['Qty'].sum()   .mean()  .size()  .sort_values()
COUNT    df['Prod'].value_counts()   .idxmax()   .max()   .apply(lambda x: ..)
WRITE    df.to_csv('out.csv', index=False)   open('f.txt','w').write(..)
NUMPY    np.array([..])  .shape  a[1:4]  m[r,c]  m[:,0]  a[a>5]  a[[0,2]]
         a.sum()/mean()/min()/max()   m.sum(axis=0) per column, axis=1 per row
TRAPS    parse_dates needs dayfirst=True | use & not and
         text column -> 'Unknown', number column -> number
```